

30 MIN TRAINING

Agent Skills

把重复经验变成可复用能力

从概念、生态、结构到部署与验证

你将带走

一张组件选择图、一套 Skill 结构、一条创建与验证闭环。

适合对象

第一次接触 Agent Skills，或希望把反复任务标准化的团队。

30 分钟路线图：先会选，再会做

0-5 min	5-12 min	12-21 min	21-27 min	27-30 min
为什么需要 Skill 的价值与渐进式披露	怎么组合 Tools / MCP / Subagents	怎么创建 结构、触发、资源与案例	怎么部署 Desktop / API / CLI / SDK	怎么验收 质量门与行动清单

学完能做到

01 选对组件 判断一个需求应该用 Prompt、Tool、MCP、Subagent 还是 Skill。	02 写出可触发结构 用 name + description 让智能体在正确场景加载正确说明。	03 建立验证闭环 检查输入、输出、异常、权限和视觉质量，而不是 “能跑就算完” 。
--	--	--

源码范围：README + 10 章正文 + 第 6 章 7 个示例/参考 Markdown，共 18 个文件。

讲解建议：先说明这不是语法课，而是把重复工作工程化的方法课。

Skill 是文件夹，不是更长的 Prompt

一个轻量、开放的能力包：把专业指令、可执行脚本和输出资源组织到同一目录。

Instructions · 指令

告诉智能体任务目标、输入输出、顺序、约束和验收标准。

Scripts · 脚本

把脆弱、重复、需要确定性的步骤固化成可执行逻辑。

Assets / References

模板与数据用于产出；领域知识和规则按需读取。

核心判断

如果某个工作流会被反复解释、反复打包资料、反复要求一致输出，就值得做成 Skill。

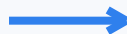
适用：领域知识 · 可重复流程 · 新能力（文档、表格、PDF、研究、代码等）

从一次性对话，到可维护的数字资产

没有 Skill

- 每次重述相同步骤
- 每次重新附上规则与模板
- 输出结构随对话漂移
- 经验只留在个人脑中
- 难以测试、审查与版本控制

工程化



有 Skill

- 指令、脚本和资源一次组织
- 触发时自动加载所需知识
- 相同输入获得更稳定的流程
- 可在团队与多个环境中复用
- 可通过测试持续迭代

本质：把隐性经验显性化，把“人会做”变成“系统可重复执行”。

渐进式披露：只在需要时加载



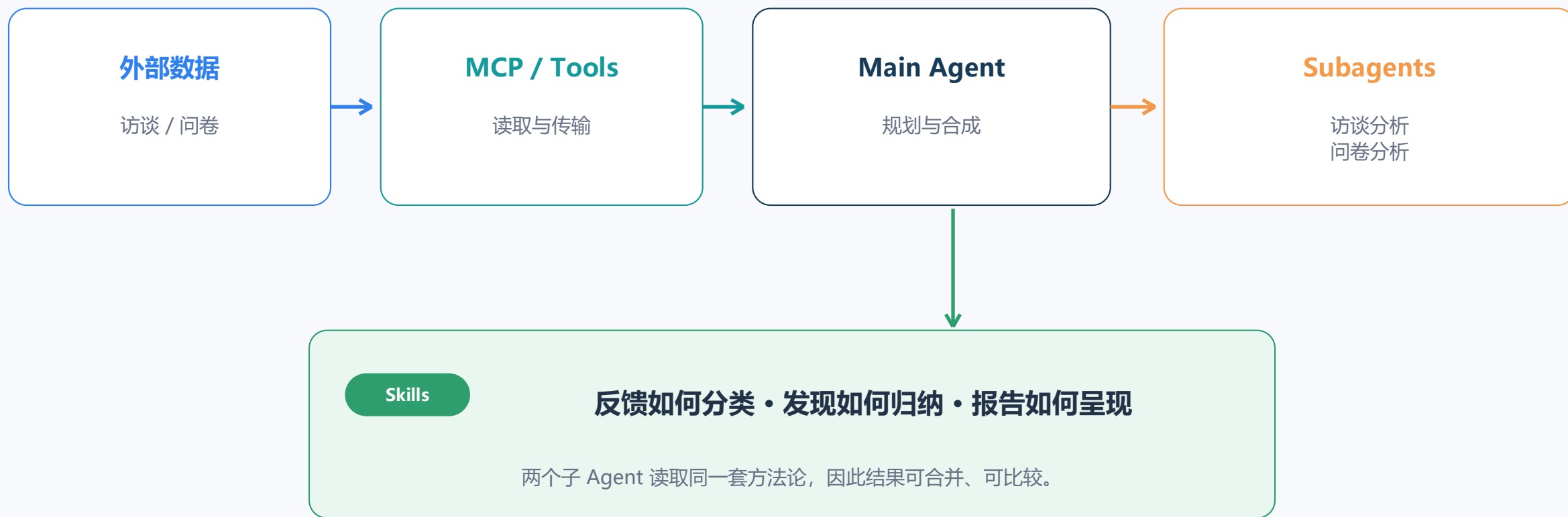
Prompt、Tool、MCP、Subagent、Skill 怎么选

组件	它解决什么	最适合	不要把它当成
Prompt	表达本次意图	一次性请求、临时偏好	长期知识库
Tool	执行底层动作	读写文件、运行命令、调用函数	完整工作流
MCP	连接外部系统	数据库、云盘、Notion 等实时数据	处理数据的方法论
Subagent	隔离或并行上下文	研究、审查、多个独立任务	可复用知识本身
Skill	封装可重复 SOP	领域规则、稳定流程、模板与脚本	外部连接器

口诀：MCP 带来数据，Tool 采取动作，Subagent 分开工作，Skill 教它按什么标准做。

讲解建议：先问“缺的是指令、动作、外部数据还是隔离上下文”，再选组件。

组合案例：客户洞察分析器



最终输出：统一、结构化、可追溯的客户洞察报告

一个可维护 Skill 的最小骨架

```
my-skill/  
├─ SKILL.md # 必需入口  
├─ agents/  
│   └─ openai.yaml # UI 元数据  
├─ scripts/ # 确定性执行  
├─ references/ # 按需知识  
└─ assets/ # 模板与数据
```

边界原则：同一份知识只放一个 owner，避免正文与参考资料重复。

讲解建议：讲清必需入口与三类可选资源，强调文件各有单一职责。

SKILL.md

Frontmatter 只放发现所需的 name 与 description；正文放执行说明。

References & Assets

规则、数据字典进 references；模板、样例工作簿和图像进 assets。

Scripts

重复且易错的计算、转换、验证步骤用脚本固定，并实际运行测试。

触发设计：名字负责识别，描述负责决策

Name

课程建议使用动词 + ing；小写字母、数字与连字符；短、准、与目录一致。

例：analyzing-time-series

Description = 做什么 + 何时用

写入任务、输入、输出和触发关键词。让智能体不打开正文，也能判断是否应加载这个 Skill。

差

“Analyze data.”

问题：没有数据类型、目标、使用场景或输出。

好

“Analyze CSV time series before forecasting...”

包含 stationarity、seasonality、trend、forecastability 等触发词。

快速自测：看到用户请求时，仅凭 description 能否回答“该不该用我”？

资源分层 + 合适的自由度

High freedom

多种方法都可行：写原则、判断标准和示例。

Medium freedom

有首选模式：给伪代码、参数和可调整步骤。

Low freedom

步骤脆弱：引用已测试脚本，明确顺序和失败处理。

更灵活

更确定

资源	放什么	设计提示
scripts	处理、转换、验证代码	写清依赖和错误；说明“执行”还是“阅读”
references	规则、算法、数据字典	单文件聚焦；较长文档顶部加目录
assets	模板、图像、数据文件	作为输出素材，不要塞进上下文

课程建议：SKILL.md 控制在约 500 行内；引用尽量保持一级深度。

讲解建议：把任务的容错空间与说明精度对应起来；越脆弱，约束越具体。

创建闭环：从例子出发，验证后迭代



最重要的顺序

具体使用例子 → 可复用资源 → 简洁 SKILL.md → 运行与视觉验证

不要从“想写一篇很完整的说明文档”开始。

案例 1：根据讲义生成练习题



为什么要拆资源

- SKILL.md** 控制问题结构与质量标准
- examples_by_topic.md** 按机器学习主题给示例
- markdown_template.md** 控制最终版式，不污染主上下文
- 可复用启示** 规则、示例、模板各归其位

讲解建议：借这个案例说明模板、领域示例与主流程如何分层。

案例 2：时间序列诊断



职责分离

脚本计算统计量；参考文档定义阈值与解读；Skill 负责顺序和交付。

好处：统计结果可复现，解释标准可审查，主说明保持简洁。

案例 3: Excel 分析 = 公式 + 规则 + 验证

课程中的营销分析

CTR = 点击 / 展示

CVR = 转化 / 点击

ROAS = 收入 / 花费

CPA = 花费 / 转化

Skill 的通用拆法

- ① 数据质量：空白、负数、异常关系
- ② 公式：可追溯、无硬编码
- ③ 规则：按顺序分类, first match wins
- ④ 输出：表格、解释、预算/行动建议
- ⑤ 验证：总计、错误、视觉检查

本次配套

processing-excel-for-beginners

用“学习打卡”替代营销数据：同样练习保留原表、查空白/重复、加简单公式、做汇总和图表。

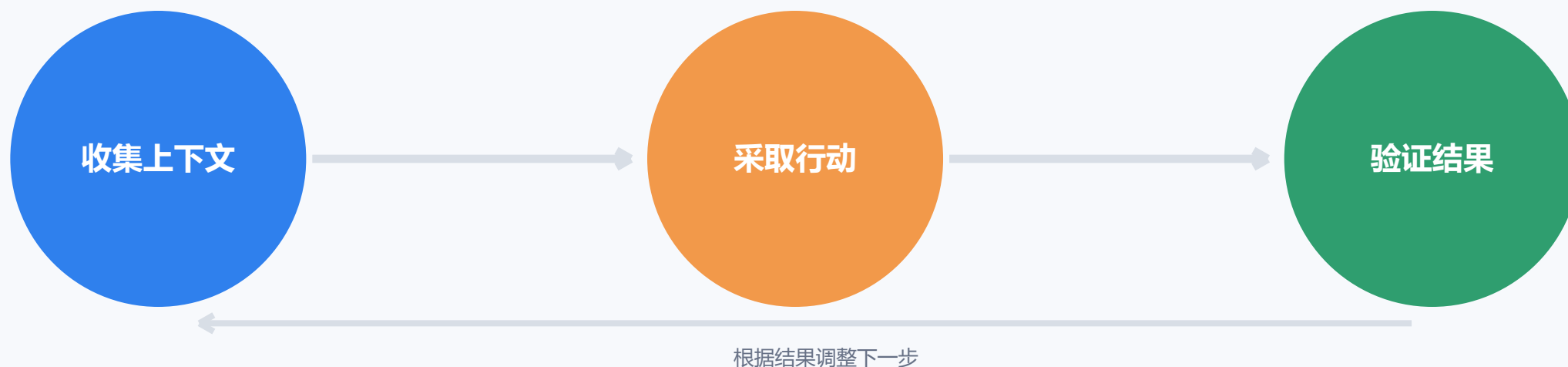
同一 Skill，不同运行表面

表面	平台通常提供什么	适合	关键提醒
Desktop / AI	文件与代码执行开箱即用	快速验证、办公产出	界面与能力随版本变化
Messages API	需显式启用代码执行与 Files API	产品化、自动调用	上传/下载通过文件 ID
Claude Code / CLI	项目、终端、Git、CLAUDE.md	编码与本地工作流	收集上下文 → 行动 → 验证
Agent SDK	可编程工具集、SkillTool、TaskTool	复杂应用与编排	显式权限与错误处理

课程提醒：不同表面不会自动共享已安装的 Skills；建议用版本库维护同一来源。

产品菜单、模型名和接口参数具有时效性，请以目标平台当期官方文档为准。

Agent 循环与安全边界



最小权限

只给完成任务所需的工具；子 Agent 的权限独立配置。

Human-in-the-loop

删除、覆盖、发布、外部写入等高风险动作先暂停确认。

错误要可见

不要用宽泛 fallback 吞掉失败；保留上下文并报告位置。

上线前质量门：八项检查

01

触发

name 清晰, description 覆盖真实请求

02

输入

格式、字段、缺失和前置条件明确

03

输出

结构、文件名、单位与可见字段稳定

04

分层

正文简洁, 详细知识按需读取

05

脚本

依赖清楚, 实际运行, 错误有帮助

06

边界

不猜数据, 不静默覆盖, 不吞异常

07

验证

公式、总计、测试和视觉检查通过

08

安全

最小权限, 高风险动作有人确认

通过标准：另一个人拿到 Skill，不需要作者在旁边解释，也能安全地产出可核对结果。

收束：先封装一个高频、小而清晰的流程

1. 你反复解释什么？

找一个每周都会说一遍的任务，而不是先做“大而全平台”。

2. 哪一步最容易错？

把确定性、易错或昂贵的步骤做成脚本和检查。

3. 怎样算完成？

写出可观察的输出、测试、权限边界和回滚方式。

五句带走

Skill 是可复用 SOP，不是超长 Prompt。
MCP 提供外部连接，Subagent 提供隔离与并行。
description 决定能否被正确触发。
脚本、参考、资产各归其位。
验证与安全是能力的一部分。

课后 10 分钟练习：把一个重复任务写成“输入 → 步骤 → 输出 → 验收”四行草稿。

附录 A：18 个 Markdown 的内容覆盖

源码文件	培训页	吸收内容
README.md	1-2	项目定位、10 章路线与受众
1. Introduction	1-3	定义、开放标准、组合方式
2. Why Use Skills 1	3-5, 14	价值、渐进式披露、Excel 案例
3. Why Use Skills 2	4, 7	Agent 能力与领域知识
4. Skills vs Tools / MCP / Subagents	6-7	组件边界与客户洞察案例
5. Exploring Pre-Built Skills	6, 15	预制 Skills、Office 与本地入口
6. Creating Custom Skills	8-14, 17	命名、结构、资源、自由度、评测
7. Skill with Claude API	15-16	代码执行、Files API、容器
8. Skill with Claude Code	15-16	Agent 循环、上下文、扩展层
9. Skills with Agent SDK	7, 15-16	主/子 Agent、MCP、最小权限
10. Conclusion	18	工程化思维与课程收束

讲解建议：用覆盖表审查培训是否真正基于全部课程文件。

附录 B：示例 Skill 与参考材料覆盖

文件	培训页	吸收内容
analyzing-marketing-campaign/SKILL.md	14	输入字段、质量检查、漏斗与效率指标
budget_reallocation_rules.md	14, 17	规则顺序、阈值、预算约束与输出表
analyzing-time-series/SKILL.md	13	诊断 → 可视化 → 报告三步 workflow
interpretation.md	13	平稳性、季节性、趋势、可预测性解释
generating-practice-questions/SKILL.md	12	输入提取、四类题型、质量标准
examples_by_topic.md	12	按主题组织领域示例与难度设计
markdown_template.md	10, 12	模板归 assets；输出结构与格式规范

来源：<https://github.com/datawhalechina/agent-skills-with-anthropic> · commit 602b1843066d · 全部内容已做摘要式转述。